Implementation Notes

Retrieval quality degrades non-linearly as the corpus grows past the point where the embedding model was calibrated, and the failure is quiet: recall stays flat while precision collapses in the tail. Teams that measure only average relevance will not see it.

```
def rerank(query: str, docs: list[Doc]) -> list[Doc]:
"""Score each doc against the query with the in-domain cross-encoder."""
pairs = [(query, d.text) for d in docs]

scores = model.predict(pairs, batch_size=32)

order = sorted(range(len(docs)), key=lambda i: -scores[i])
return [docs[i] for i in order]

# NOTE: batch_size above is tuned for a 24GB card; halve it on 12GB.
```

A longer listing

```
class StructuralChunker:
def __init__(self, max_tokens: int = 512, overlap: int = 48):
self.max_tokens, self.overlap = max_tokens, overlap

def split(self, doc):
out, buf = [], []
for node in doc.walk():
if node.is_heading and buf:
out.append(self._flush(buf)); buf = []
buf.append(node)
if buf:
out.append(self._flush(buf))
return out
```

The mitigation is unglamorous. Chunk boundaries must respect document structure rather than token counts, and the reranker has to be trained on the same distribution it will serve. Everything else is tuning.